Klausur Media Programming: Keywords (AV + Graphics)

> Klausur: **08.10.2026, 11:00–12:30**
> Hình thức: không phải viết Python đúng cú pháp. Cần **đọc hiểu code, điền chỗ trống, viết pseudo-code**, **tính toán** (máy tính bỏ túi), **chọn và biện luận** (format, công cụ).
> ⭐ = rất hay ra thi / đã sai trong Test

PHẦN A: AV PROGRAMMING (Prof. Wilk)

Thema 1: Audio-Signalweg & digitale Repräsentation

Chuỗi tín hiệu:

```
Schall → Mikrofon → ADC → Audio-Interface → WAV → Python-Array → DAC → Lautsprecher
```

- Chuyển từ analog sang digital xảy ra ở **ADC**.
- Chuyển từ digital ngược về analog xảy ra ở **DAC**.

Keyword	Ý nghĩa
Abtastung (Sampling)	Rời rạc hóa theo **thời gian**
Abtastfrequenz / Samplerate	Số mẫu mỗi giây, ví dụ 8000 / 16000 / 44100 / 48000 Hz
Quantisierung	Rời rạc hóa theo **biên độ**. Không đảo ngược được, luôn sinh ra **Quantisierungsfehler**
Bittiefe / Wortbreite	Mức chia biên độ mịn đến đâu: 16 bit = 2^{16} mức. Dynamik ≈ **6 dB × Bit** (16 bit ≈ 96 dB, 8 bit ≈ 48 dB)

Keyword	Ý nghĩa
⭐ **Nyquist-Kriterium**	Phải có **fs > 2·f_max**. Nếu không sẽ bị **Aliasing**. Vì vậy cần **bandbegrenzung** (lọc thông thấp) trước khi lấy mẫu
PCM	Pulse Code Modulation: các giá trị mẫu được lưu trực tiếp dưới dạng nhị phân
WAV	**Container** dựa trên **RIFF** (Microsoft). Có chunk `fmt` (tham số) và chunk `data` (dữ liệu PCM thô). `fmt` phải đứng trước `data`
Mono / Stereo	Mono là array **1D**: `(N,)` . Stereo là array **2D**: `(N, 2)`
Mono-Mischung vs. linker Kanal	Mono-Mischung = `mean(axis=1)` , tức trộn cả hai kênh. "Chỉ kênh trái" = `[:, 0]` , nghĩa là kênh phải bị bỏ đi
Zeitbereich / Frequenzbereich	Miền thời gian: giá trị thay đổi theo thời gian. Miền tần số: tín hiệu chứa những tần số nào
DFT / FFT	FFT là thuật toán nhanh để tính DFT. Với tín hiệu thực dùng `np.fft.rfft` , `rfftfreq`
STFT	FFT trên các cửa sổ thời gian ngắn, nên thấy được **tần số thay đổi theo thời gian** (spectrogram). Một FFT đơn lẻ chỉ cho biết có những tần số nào, không cho biết lúc nào

⭐ **Đừng nhầm Samplerate với Bittiefe:**

- **Samplerate** quyết định **dải tần biểu diễn được**. Samplerate thấp thì mất tần số cao, tiếng nghe **đục/dumpf**.
- **Bittiefe** quyết định **Quantisierungsfehler** và dải động. Bittiefe thấp thì có **nhiễu/rè**, nhất là ở đoạn nhỏ tiếng.

⭐ **Công thức:**

```
Datenrate [bit/s]  = fs × Bittiefe × Kanäle
Dateigröße [Byte]  = Dauer × fs × Kanäle × (Bittiefe / 8)    ← chia 8, KHÔNG chia 2!
Dauer              = frames / samplerate
```

Ví dụ: 10 s, 44100 Hz, Stereo, 16 Bit → 10 × 44100 × 2 × 2 = **1.764.000 Byte**.

Code tạo sóng sin:

```
t = np.linspace(0, dauer, int(sr * dauer), endpoint=False)
signal = 0.5 * np.sin(2 * np.pi * f * t)
audio = (signal * 32767).astype(np.int16)
write("x.wav", sr, audio)        # scipy.io.wavfile
```

Đọc metadata với module `wave` : `getnchannels()` , `getsampwidth()` (đơn vị **Byte**), `getframerate()` , `getnframes()` .
File thật lớn hơn một chút so với kết quả tính vì có thêm **Header**.

Thema 2: Video-Signalweg

Chuỗi tín hiệu:

```
Szene → Kamera/Optik → Bildsensor → Auslesen/Digitalisierung → Frame/Datei → Python
      → Speicher → Grafik-API → Display-Controller → Monitor
```

- **Bayer-Matrix:** cảm biến chỉ đo độ sáng, màu có được nhờ lớp lọc màu. Trong mỗi nhóm 4 pixel có **2 Grün, 1 Rot, 1 Blau**, vì mắt người nhạy nhất với màu xanh lá.
- **fps:** cảm nhận được chuyển động từ khoảng **16 fps**. **24 fps** là chuẩn điện ảnh. 50/60 fps cho hình mượt hơn.
- **Độ phân giải:**
  - HD: 1280×720
  - Full HD: 1920×1080
  - **UHD (4K):** 3840×2160
  - **DCI 4K (rạp phim):** 4096×2160
  - 8K: 7680×4320
- ⭐ **OpenCV dùng BGR, không phải RGB:** kênh `[:,:,0]` = **B**, `[:,:,1]` = G, `[:,:,2]` = **R**.
- ⭐ `frame.shape` **= (Höhe, Breite, Kanäle)**, ví dụ `(720, 1280, 3)` . Ảnh xám có shape `(720, 1280)` .
- ⭐ Chuyển sang ảnh xám: `cv.cvtColor(frame, cv.COLOR_BGR2GRAY)` . Quy tắc đặt tên là `COLOR_VON2NACH` .

OpenCV API:

```
cap = cv.VideoCapture("video.mp4")   # 0 = Webcam
cap.isOpened()
ret, frame = cap.read()              # ret == False → hết video
cap.get(cv.CAP_PROP_FPS)
cap.get(cv.CAP_PROP_FRAME_COUNT)
cap.get(cv.CAP_PROP_FRAME_WIDTH)
cap.get(cv.CAP_PROP_FRAME_HEIGHT)
cap.get(cv.CAP_PROP_POS_FRAMES)      # vị trí theo frame
cap.get(cv.CAP_PROP_POS_MSEC)        # vị trí theo mili-giây
cv.imwrite(...)
cv.imshow(...)
cv.waitKey(1)
cap.release()
cv.destroyAllWindows()
```

- ⭐ **Dauer = frame_count / fps**. Chỉ số thời gian của một frame: **t = frame_idx / fps**.
- "Lưu mỗi frame thứ 30" chỉ bằng 1 giây khi video có 30 fps. Cách này dựa trên frame, **không dựa trên thời gian**.

Thema 3: Audioformate & Datenreduktion

⭐ **Container và Codec:**

- **Container** tổ chức cách dữ liệu nằm trong file. Ví dụ: WAV, Ogg, Matroska (.mka/.mkv), MP4/M4A.
- **Codec** quyết định dữ liệu được mã hóa/nén như thế nào. Ví dụ: PCM, FLAC, MP3, AAC, Vorbis.
- Ví dụ ghép: `musik.wav` = WAV + PCM, `podcast.m4a` = MP4 + AAC, `x.ogg` = Ogg + Vorbis.
- ⭐ Câu "WAV ist unkomprimiert" là **không chính xác**. Câu đúng: "WAV **enthält (üblicherweise) PCM-Daten**", vì WAV là container.

Format	Loại	Điểm mạnh	Điểm yếu	Dùng cho
WAV/PCM	không nén	minh bạch, đọc thẳng vào NumPy	rất lớn	phân tích, thu âm chuyên nghiệp
FLAC	**lossless**	nhỏ hơn WAV 40–60%, khôi phục **bit-chính xác**, mở, có metadata	ít phổ biến hơn WAV	**lưu trữ (Archivierung)**

Format	Loại	Điểm mạnh	Điểm yếu	Dùng cho
MP3	lossy, perceptual	phổ biến nhất	mất chất lượng	phát hành (Distribution)
AAC	lossy, perceptual	**hiệu quả hơn MP3**	mất chất lượng	streaming, podcast

- **Lossless (FLAC):** dùng **Prädiktor-Korrektor**. Encoder dự đoán mẫu tiếp theo rồi chỉ truyền **sai số e**. Decoder dự đoán giống hệt rồi cộng e vào.
- **Lossy (MP3/AAC):**
  - phân tích thời gian/tần số
  - **mô hình tâm lý âm học (psychoakustisches Modell)**
  - **Verdeckung/Maskierung**: nhiễu lượng tử hóa được phép lớn đến mức vẫn bị âm khác che, trong cùng **Frequenzgruppe**
  - lượng tử hóa và mã hóa
  - **Joint Stereo**
- **Bitrate thấp** thì file nhỏ hơn nhưng chất lượng kém hơn. Ví dụ `-b:a 128k` so với `64k`.
- **ffmpeg** dùng để chuyển đổi. **ffprobe** dùng để phân tích (container, streams, codec, bitrate).
- Gọi từ Python:
  - `subprocess.run([...], capture_output=True, text=True)`
  - `shutil.which("ffprobe")` : kiểm tra chương trình có trong PATH không
  - `json` : đọc output của ffprobe khi dùng `-print_format json`

Mẫu câu trả lời khi chọn format (luôn có: Format + 2 lý do + 1 nhược điểm):

- **Phân tích bằng Python → WAV.** Lý do: lossless, đọc thẳng vào NumPy không cần decode. Nhược điểm: file lớn.
- **Lưu trữ → FLAC.** Lý do: lossless và nhỏ hơn WAV, format mở/miễn phí bản quyền, có metadata. Nhược điểm: ít phổ biến hơn WAV.
- **Podcast/streaming → AAC (hoặc MP3).** Lý do: nhỏ, phổ biến. Nhược điểm: lossy.

Thema 4: Videoformate & Datenreduktion

- **Container video:** MP4 (phổ biến nhất), **MKV** (mở, chứa được nhiều track), OGG.
- **Codec video:**

Codec	Năm	Hiệu quả	Bản quyền
H.264/AVC	2003	chuẩn cơ sở	trả phí
H.265/HEVC	2013	tốt hơn H.264 khoảng 2 lần, dùng cho 4K/HDR	trả phí
AV1	2018 (AOMedia)	tốt hơn khoảng 2 lần	**royalty-free**

- **Muxer** đóng gói các stream vào container. **Demuxer** mở container và tách stream ra.
- **YCbCr / YUV:**
  - **Y** = Luma (độ sáng), **Cb/Cr** = Chroma (độ lệch màu)
  - ⭐ `Y = 0.299R + 0.587G + 0.114B` , `Cb = (B−Y)·0.564` , `Cr = (R−Y)·0.713`
  - Mắt nhạy với **độ sáng hơn màu**, nên có thể giảm Chroma mạnh hơn.
- ⭐ **Chroma Subsampling:**
  - **4:4:4:** không giảm
  - **4:2:2:** giảm một nửa theo chiều ngang
  - **4:2:0:** 4 pixel trong ô 2×2 dùng chung một cặp Cb/Cr. Dữ liệu màu giảm 75%, tổng giảm từ 24 xuống 12 bit/pixel (tức 50%)
  - `yuv420p` : p = **planar** (mỗi kênh lưu thành một khối riêng)
- **Redundanz:**
  - **räumlich** (không gian): pixel cạnh nhau trong một frame thường giống nhau
  - **zeitlich** (thời gian): các frame liên tiếp hầu như không đổi
- **Các bước nén:**
  i. **Intra-Prädiktion**: dự đoán từ block bên cạnh trong cùng frame
  ii. **Inter-Prädiktion** + **Bewegungskompensation**: dự đoán từ frame khác, lưu **Bewegungsvektor**
  iii. **Transformation (DCT)**
  iv. **Quantisierung**: ⭐ bước **duy nhất** gây mất dữ liệu
  v. **Entropiecodierung**: lossless
  vi. **Farbunterabtastung** (Chroma Subsampling)
- ⭐ **Loại frame:**
  - **I-Frame** (Intra, keyframe): độc lập hoàn toàn
  - **P-Frame**: dự đoán từ frame **trước**
  - **B-Frame**: dự đoán từ frame **trước và sau**, hiệu quả nhất
- **CRF:** 0 = chất lượng cao nhất, **23 = mặc định của H.264**, 51 = tệ nhất. CRF càng cao thì lượng tử hóa càng mạnh và file càng nhỏ.

- **In-Loop-Filter:** lọc frame đã tái tạo trước khi dùng làm tham chiếu, để giảm viền block và Ringing.
- **SDR vs HDR:**
  - SDR khoảng 100 nit, 8 bit
  - HDR 1000–10000 nit, 10/12 bit
  - **Cả chuỗi** (Kamera, Codec, Übertragung, Display) đều phải hỗ trợ HDR
- **Coding-Artefakte:**
  - **Blockartefakte** (thấy rõ ô vuông)
  - **Banding** (dải màu thô thay vì chuyển màu mượt)
  - **Detailverlust** (mất chi tiết nhỏ)
  - **Ringing** (quầng mờ quanh cạnh sắc)
- **Tính dung lượng video thô:** `B × H × 3 Byte × fps × Dauer` .
  Ví dụ 640×360, 30 fps, 10 s → khoảng **207 MB**. Sau khi nén AV1 còn khoảng 1 MB, tức nén khoảng **200:1**.

Thema 5: AV-Pipeline & Synchronisation

Pipeline:

```
Quelle → Container + Streams → Demuxing → Decoding → Analyse/Bearbeitung → Encoding → Muxing → A
```

- ⭐ **OpenCV luôn đứng SAU bước Decoding**, nó chỉ làm việc với frame đã giải mã.
- **Streams:** Video, Audio, Untertitel.
- **Stems:** Dialog, Musik, Effekte (SFX), Atmo. **M&E-Spur** = Music & Effects, không có lời thoại, dùng cho bản lồng tiếng quốc tế.
- **`-map`：** `-map 0:v:0` = video stream đầu tiên của input 0. `-map 0:a:1` = audio stream thứ hai.
- **Đơn vị tương ứng:** Video có Frame và fps. Audio có Sample và Samplerate.
- **Timestamp (PTS):** thời điểm phát của mỗi frame hoặc block.
  - `setpts=PTS-STARTPTS` : bắt đầu lại từ 0
  - `setpts=0.5*PTS` : nhanh gấp 2
  - `setpts=2.0*PTS` : chậm 2 lần
  - `asetpts=N/SR/TB` : tạo lại timestamp audio từ số sample
- ⭐ **Ba loại lỗi đồng bộ:**
  - **Startversatz / Offset:** lệch **cố định** ngay từ đầu, ví dụ track thu ngoài bị đặt sai vị trí
  - **Drift:** đầu khớp, sau đó lệch **tăng dần**, vì Samplerate/Framerate thực khác với giá trị khai báo
  - **Instabilität / Schwankung:** lệch dao động không đều, do buffer overflow hoặc I/O-Jitter

- ⭐ **Tổng thời lượng đúng KHÔNG có nghĩa là đồng bộ đúng**, vì tổng thời lượng không nói gì về timestamp bên trong.
- **Latenz chấp nhận được:**
  - phát lại file: không quan trọng
  - Live-Streaming: dưới 2–5 giây
  - Video-Call: dưới 150 ms
  - Gaming: dưới 50 ms
- **Tách audio cho Speech-to-Text:** `ffmpeg -i in -map 0:a:0 -ac 1 -ar 16000 out.wav` (Mono, 16 kHz).
  Dùng cho: Transkription, phân tích mức âm/khoảng lặng, kiểm tra chất lượng (Clipping), Feature (RMS, ZCR, MFCC).
- ⭐ **Remuxing, Transcoding, Schneiden:**
  - **Remuxing** (`-c copy`): đổi container, giữ nguyên codec, rất nhanh, không mất chất lượng
  - **Transcoding** (`-c:v libx264`): đổi codec, chậm, có thể mất chất lượng
  - **Schneiden** (`-ss` , `-t` , `-to`): cắt theo thời gian
  - ⭐ Cắt bằng `-c copy` **không chính xác đến từng frame**, vì nó bắt đầu từ keyframe gần nhất. Muốn chính xác thì phải encode lại.

Thema 6: AV-Bearbeitung & Synchronisation

- **ITU-T J.248** là chuẩn về giám sát độ trễ giữa video và audio (Lip-Sync).
  - **Ngưỡng nhận biết (Wahrnehmungsgrenzen): +45 ms** khi Audio đi trước Video, **−125 ms** khi Audio đi sau Video
  - **Ngưỡng chấp nhận (Akzeptanzgrenzen):** +90 ms đến −185 ms
  - Với sự kiện ngắn, đột ngột (impulshaft), ngưỡng còn chặt hơn
  - ⚠️ Từ các con số trên: **audio đi trước** bị nhận ra sớm hơn (chỉ 45 ms). Điều này hợp lý vì trong tự nhiên âm thanh không bao giờ đến trước hình ảnh.
  - Nguyên nhân chính: **trễ phía video** (Frame-Synchronizer, Encoder/Decoder, xử lý ảnh).
- **Sửa Offset** (dịch timestamp):
  - `adelay=500:all=1` : làm audio trễ 500 ms
  - `atrim=start=… + asetpts=PTS-STARTPTS + apad` : cắt phần đầu audio, bù khoảng lặng ở cuối
  - `-itsoffset 0.5 -i INPUT` : làm trễ toàn bộ input
  - `setpts=PTS+0.08/TB` : video trễ 80 ms (chỉ dịch timestamp, không đổi nội dung hình)
- **Sửa Drift** (co giãn tốc độ):
  - `atempo=Faktor` , ví dụ Faktor = 59,65 / 60,00 = **0,9942**
  - `aresample=async=1000` : kéo giãn/nén audio cho khớp timestamp
- ⭐ **Quy tắc vàng:** Offset thì dịch timestamp (`setpts` / `asetpts` / `adelay`). Drift thì đổi tốc độ (`atempo` , `aresample`).

- **Bộ lọc audio của ffmpeg:**
  - `volume` : chỉnh âm lượng
  - `silenceremove` : bỏ đoạn quá nhỏ
  - `highpass` : giảm nhiễu tần thấp
  - `lowpass` : giảm nhiễu tần cao
  - `amix=inputs=2:duration=first` : trộn nhiều nguồn audio (duration có thể là first/longest/shortest)
- ⭐ **Đổi dB sang hệ số tuyến tính: gain = 10^(dB/20)**, ví dụ +6 dB ≈ ×2.
- **RMS** = `sqrt(mean(x²))` tính trên mỗi khung khoảng 20 ms.
  - **Gate:** khung có RMS dưới ngưỡng thì nhân với 0 (hoặc giảm). **Cấu trúc thời gian được giữ nguyên.**
  - **Schneiden (cắt):** bỏ hẳn khung dưới ngưỡng, nên file **ngắn lại**.
- **Crossfade:** `fade_out = linspace(1,0,n)` , `fade_in = linspace(0,1,n)` , sau đó cộng hai đoạn chồng lên nhau. Hai file **phải có cùng Samplerate**.
- **OpenCV:**
  - `GaussianBlur(frame, (15,15), 0)` : kernel phải có kích thước **lẻ** để có tâm rõ ràng
  - `addWeighted(f1, α, f2, β, 0)` : chuyển cảnh (Überblendung). Hai video cần cùng độ phân giải và cùng fps
  - `VideoWriter` : ghi video ra file
  - `np.clip(…, 0, 255)` : chống tràn giá trị (Clipping)
- **Bearbeitungseffekte vs. Analyseoperationen:**
  - Bearbeitungseffekte **thay đổi** nội dung (gain, highpass, blur, graustufen…)
  - Analyseoperationen **đo/đánh giá**, không đổi nội dung (đo thời lượng, đánh giá offset/drift…)
- **Nguồn lỗi thường gặp:**
  - Clipping khi tăng âm lượng
  - ngưỡng cắt quá gắt
  - **nhầm Offset với Drift**
  - chỉ sửa phần đầu mà không xử lý quá trình (không sửa Drift)
  - nhầm BGR/RGB
  - xuất sai khoảng thời gian / không tính đúng fps

Thema 7: Audio- & Video-Inhaltsanalyse

Mô hình cơ bản:

```
Rohdaten → Merkmale (Features) → Inferenz → Anwendung
```

⭐ Rohsignal chưa phải là nội dung. Merkmale chỉ mô tả **cấu trúc**, chưa mô tả **ý nghĩa**.

Ebene	Audio	Video
Low-Level	RMS, Spektrum, spektrale Helligkeit	Farbe, Frame-Differenz
Mid-Level	Pitch-Verlauf, Beat, Segmentgrenzen	Shots, Keyframes, Tracking
High-Level	"vermutlich Musik"	"Shot-Wechsel", "Highlight"

- **5 ứng dụng:**
  i. Podcast-Segmentierung
  ii. Tempo-/Beat-Erkennung
  iii. **Audio-Fingerprinting** (nhận diện bằng profile đặc trưng gọn, chịu được nhiễu)
  iv. **Shot-Erkennung**
  v. Objekt-/Aktivitätserkennung
- **Onset:** điểm bắt đầu của một âm. **Onset-Stärke** là đường cong theo thời gian, các đỉnh là onset có thể có.
- **librosa:**
  - `feature.rms`
  - `feature.spectral_centroid`
  - `yin` (ước lượng **F0**)
  - `beat.beat_track` (trả về tempo theo **BPM** và thời điểm beat)
- ⭐ **Spectral Centroid** = `Σ|X[k]|·f[k] / Σ|X[k]|` . Đây là thước đo **độ "sáng" của phổ**, **KHÔNG phải cao độ (Pitch)**.
- **F0 bằng YIN:** tìm độ trễ mà tại đó tín hiệu lặp lại tuần hoàn, rồi đổi chu kỳ sang Hz.
- **Chuỗi xử lý Beat:** Onset-Stärke → Beat-Tracking → Tempo-Schätzung.
- ⭐ **Shot so với Bewegung** (rất hay ra thi):
  - **Histogrammdifferenz** (`calcHist` trên HSV, `compareHist` BHATTACHARYYA): đo thay đổi phân bố màu tổng thể, phù hợp để phát hiện **cắt cảnh (Schnitt)**
  - **Frame-Differenz** (`absdiff` trên ảnh xám rồi lấy `.mean()`): đo thay đổi từng pixel, phù hợp để đo **mức chuyển động**
  - Người đi qua phòng: Histogram thay đổi ít, Frame-Differenz thay đổi nhiều
  - Cắt cảnh: cả hai đều tăng mạnh
  - Lưu ý: chuyển động máy quay hoặc đổi ánh sáng cũng làm Frame-Differenz tăng cao
- **Klassisch vs. ML:**
  - Phương pháp cổ điển: minh bạch, nhanh, nhưng sức mô tả nội dung hạn chế
  - ML: mạnh hơn, nhưng **Black-Box** và phụ thuộc vào Trainingsdaten
- **YAMNet:** 521 lớp (AudioSet). Input là **Mono 16 kHz, chuẩn hóa về [−1, 1]**. ⭐ Scores **không phải xác suất**, tổng không bằng 1.

- **Faster R-CNN (torchvision):** trả về Bounding-Box + Klasse + Konfidenz. `score_threshold` thấp hơn thì hiện nhiều kết quả hơn nhưng kém chắc chắn hơn.

FFmpeg Hilfsblatt (tóm tắt nhanh)

```
ffmpeg [Eingabeoptionen] -i IN [Ausgabeoptionen] OUT     ← vị trí option quan trọng!
ffprobe -v error -show_format -show_streams IN           (-select_streams v|a)

-map 0:v:0 -map 1:a:0     # video từ input 0, audio từ input 1
-an / -vn                 # bỏ audio / bỏ video
-c copy, -c:v copy        # không encode lại
-c:v libx264, -c:a aac    # encode lại
-vf "scale=1280:-2"       # đổi độ phân giải, giữ tỉ lệ khung
-vf "fps=25"              # đổi framerate
-b:v 2M, -b:a 128k        # bitrate
-ar 48000, -ac 1          # samplerate, số kênh
-af "aresample=48000"     # resample tường minh
-ss 10 -t 20              # bắt đầu từ giây 10, dài 20 giây
-frames:v 1 bild.jpg      # lấy 1 ảnh
-vf fps=1 f_%03d.jpg      # 1 ảnh mỗi giây
-filter_complex "[0:a][1:a]amix=inputs=2[a]" -map 0:v -map "[a]"
```

⭐ Khi đã áp **filter** lên một stream (scale, mix, resample…) thì stream đó **phải encode lại**, không dùng `copy` được.

PHẦN B: GRAPHICS PROGRAMMING (Prof. Langbehn)

Unit 1: Human Perception

- **Perception → Cognition → Action:** Receptors → Zentralnervensystem → Motorneuronen.
- **Licht** là sóng điện từ. **Frequenz** quyết định màu, **Amplitude** quyết định độ sáng. Giấy trắng phản xạ khoảng 90%, giấy đen khoảng 10%.
- **Retina:**
  - **Zapfen (Cones)** 7–8 triệu: màu và hình dạng

    - **Stäbchen (Rods)** 100–120 triệu: sáng/tối và chuyển động
- **Gegenfarbentheorie:** các cặp đối kháng Rot–Grün, Blau–Gelb, Schwarz–Weiß. Giải thích được **Nachbild** (ảnh dư).
- **Farbkonstanz:** não vẫn thấy màu ổn định khi ánh sáng thay đổi.
- **Vestibuläres System:**
    - **Otolithen** (Sacculus, Utriculus): cảm nhận gia tốc tuyến tính và trọng lực. Không phân biệt được nghiêng đầu với chuyển động thẳng (**GIA-Vektor**)
    - **Bogengänge:** cảm nhận chuyển động xoay
- **Haptik, Taktil** (áp lực, đau, nhiệt), **Propriozeption/Kinästhetik** (cảm nhận vị trí và chuyển động của cơ thể).
- **Sensorische Integration:**
    - **Maximum Likelihood Estimation:** mỗi kênh có trọng số theo chất lượng, **thị giác thường chiếm ưu thế**
    - **Bayes:** kết hợp thêm kiến thức có trước (prior)
- **Selektion/Filterung:** lượng thông tin vào khoảng 1,5 GBit/s, nhưng chỉ một phần nhỏ đến được ý thức. Ví dụ: **McGurk-Effekt** (hình môi làm thay đổi âm nghe được).
- **Gedächtnis (Atkinson-Shiffrin):**
    - **Sensorisches Gedächtnis** (ikonisch, echoisch, haptisch)
    - → Attention → **Kurzzeit-/Arbeitsgedächtnis**
    - → **Langzeitgedächtnis**
    - Liên quan: Cognitive Load

Unit 0/2: Encoding & Python

- **ASCII:** H = 72 = `01001000`.
- **RLE:** `AAABBBBAA` → `3A4B2A`.
- ⭐ **NumPy:**
    - Thao tác **vector hóa** trên cả array, **không dùng vòng for** qua từng pixel
    - Gán `pixel = ...` trong vòng lặp không thay đổi array gốc

Unit 3: Raster Graphics

- **Rastergrafik:** lưới pixel. Gốc tọa độ (0,0) ở **góc trên bên trái**. Truy cập pixel bằng `img[y][x]`, tức **hàng trước, cột sau**.
- **Ưu điểm:** chi tiết ảnh chụp, chỉnh được từng pixel, tương thích rộng.
- **Nhược điểm: phóng to bị vỡ hạt (Pixelation)**, file lớn, khó tách đối tượng, độ phân giải cố định.
- **So sánh format:**

Format	Nén	Trong suốt	Dùng cho
JPEG	lossy	không	ảnh chụp
PNG	lossless	có (Alpha)	logo, icon
GIF	LZW	1 bit	tối đa 256 màu, animation
WebP	lossy/lossless	có	web
TIFF	lossless	có	in ấn, lưu trữ

- **RGB** là màu cộng (additiv): (0,0,0) = đen, (255,255,255) = trắng. **RGBA** có thêm Alpha để điều khiển độ trong suốt.
- **Farbtiefe:**
  - 1 bit = 2 màu
  - 8 bit = 256 màu (indexed)
  - **24 bit = 16,7 triệu màu (True Color)**
  - 10–16 bit/kênh: HDR, ít banding hơn
- ⭐ **Dateigröße = B × H × Bittiefe / 8.** Ví dụ Full HD 24 bit = 6.220.800 Byte ≈ **5,93 MB**.
- ⭐ **Offset trong RAM** (lưu theo hàng): **Offset = (y · width + x) · (bpp / 8)**.

⭐ **JPEG-Pipeline (C → B → E → A → D):**

1. **RGB → YCbCr** (tách độ sáng khỏi màu)
2. **Chroma Subsampling** (4:2:0)
3. Chia thành **block 8×8**
4. **DCT**: chuyển miền không gian sang miền tần số. Hệ số **DC** nằm ở góc trên trái. Bước này tự nó là **lossless**
5. **Quantisierung**: `B = round(DCT / Q)` . ⭐ Đây là **bước DUY NHẤT gây mất dữ liệu**. Tần số cao bị làm tròn về 0
6. **Entropy Coding** (lossless, bước cuối): **Zig-Zag-Scan** → **RLE** (gom các số 0) → **Huffman** (giá trị càng hay gặp thì mã càng ngắn)

Vì sao JPEG hiệu quả: mắt nhạy với **độ sáng hơn màu**, và **kém nhạy với tần số cao**.

Punktoperationen (mỗi pixel xử lý độc lập):

- Brightness: `I' = I + c`
- Contrast: `I' = a·I`
- Negative: `I' = 255 - I`
- Gamma: `I' = 255·(I/255)^γ`

- Threshold: `I' = 0` nếu `I < T`, ngược lại `255`
- Histogram-Stretch: `(I − h_min) / (h_max − h_min) · w_max`

⭐ Viết code đúng (có `np.int32` để tránh tràn uint8):

```
np.int32(arr + c).clip(0, 255).astype(np.uint8)   # không có int32 → tràn uint8: 200 + 100 = 44
(255 - arr).astype(np.uint8)
```

Histogramm:

- Trục x là mức sáng từ 0 (đen) đến 255 (trắng).
- Trục y là số pixel ở mỗi mức sáng.
- Ảnh thiếu sáng (unterbelichtet) có histogram dồn về bên trái.

Filter (dùng cả pixel lân cận, tính bằng **Faltung/Convolution**):

Filter	Kernel	Ý nghĩa
Box Blur	1/9 · ma trận 3×3 toàn số 1	trung bình 9 pixel
Boost / Sharpen	tâm 9, xung quanh −1	làm nổi chi tiết
Kantendetektion (Laplace)	tâm 8, xung quanh −1	tìm cạnh
Sobel	Gx = −1,0,1],[−2,0,2],[−1,0,1, Gy là chuyển vị	**G = √(Gx² + Gy²)**

Unit 4: Algorithmic Graphics Generation

- **Image Processing vs. Image Generation:**
  - Image Processing xử lý dữ liệu đã có (ví dụ chỉnh ảnh chụp)
  - Image Generation **tạo ảnh bằng quy tắc/thuật toán**
- ⭐ **Nguyên lý cơ bản:** `I(x, y) → Farbe`.
  - `I = x`: gradient ngang
  - `I = x + y`: gradient chéo
  - `I = sin(x)`: hoa văn tuần hoàn
  - `I = √(x² + y²)`: hàm hướng tâm
- **Diskret vs. Kontinuierlich:**
  - Rời rạc: pixel là số nguyên, độ phân giải cố định
  - Liên tục: số thực, **chuẩn hóa về 0.0–1.0**
- ⭐ **Đổi giá trị toán học [−1, 1] sang [0, 255]:**

    - **Scaling** (×127,5) và **Mapping/Shifting** (+127,5)
    - **Clamping**: ép giá trị vào trong khoảng
    - **Clipping**: bỏ giá trị nằm ngoài khoảng
- **PRNG:**
    - Máy tính có tính tất định (deterministic). Cùng **Seed** thì ra cùng dãy số.
    - **Gleichverteilung** (Uniform): ví dụ tung xúc xắc
    - **Normalverteilung** (Gauss, Bell Curve): ví dụ hạt nhiễu (grain), chiều cao người
    - **Stochastische Prozesse** (Random Walk): có trạng thái/bộ nhớ, ví dụ L-System
- ⭐ **White Noise:** mỗi pixel ngẫu nhiên **độc lập**, **không có tương quan** với pixel bên cạnh, phân bố đều 0–255.
- ⭐ **Perlin Noise** (Ken Perlin, Tron 1982, Oscar 1997, Minecraft):
    - Là **kohärentes Rauschen / Gradient Noise**: các pixel lân cận có liên hệ với nhau, nên mượt và tự nhiên.
    - **4 bước:**
        a. Tạo **Grid**, gán **zufälligen Gradientenvektor** (đã chuẩn hóa) cho mỗi điểm lưới
        b. Tính **Distanzvektoren** từ 4 góc ô đến điểm đang xét
        c. Tính **Skalarprodukt** (dot product) giữa Gradient và Distanz ở mỗi góc
        d. **Interpolieren** 4 giá trị bằng **Fade-Funktion** (bilinear)
    - ⭐ **Fade:** $f(t) = 6t^5 - 15t^4 + 10t^3$. Đạo hàm bằng 0 ở hai đầu, nên không lộ cạnh tại biên ô lưới.
    - ⭐ **lerp(a, b, t) = a + t·(b − a)**. Thứ tự: lerp ngang 2 lần (cạnh trên, cạnh dưới), rồi lerp dọc 1 lần.
- ⭐ **Fraktales Rauschen (Oktaven):** $\Sigma\ Oktave_n \cdot Amp_n$.
    - Mỗi octave **Frequenz ×2**, **Amplitude ×½** (gọi là **Persistence**).
    - Mô phỏng tính **tự đồng dạng** của tự nhiên.
- **Fraktale:**
    - **Selbstähnlichkeit** (tự đồng dạng) ở mọi tỉ lệ, độ phức tạp vô hạn, **chiều phân số**
    - Ví dụ: Sierpinski-Dreieck, Pythagoras-Baum, **Koch-Kurve** (chia đoạn thẳng làm 3, thay đoạn giữa bằng một đỉnh tam giác, rồi lặp lại)
- ⭐ **Mandelbrot:**

```
c = complex(xmin + x/W*(xmax-xmin), ymin + y/H*(ymax-ymin))  # x ∈ [-2, 1], y ∈ [-1.5, 1.5]
z = 0
for i in range(max_iter):
    if abs(z) <= 2: z = z**2 + c
    else: break
image[y, x] = i
```

- `c` là **một điểm trên mặt phẳng phức**: trục x là phần thực, trục y là phần ảo.
- `i` là **Escape-Geschwindigkeit**, tức số vòng lặp đến khi `|z| > 2`. Giá trị này được tô màu qua Colormap.
- Điểm không bao giờ thoát (chạm `max_iter`) nằm **trong tập Mandelbrot** và được tô **màu đen**.

Unit 5: Vector Graphics

- **Vektorgrafik:** mô tả bằng **primitive hình học** (Linie, Kreis, Rechteck, Polygon, Kurve) thay vì pixel.
  - **Ưu điểm:** phóng to tùy ý **không vỡ hạt**, file nhỏ, dễ chỉnh từng đối tượng.
  - **Nhược điểm:** không phù hợp cho ảnh chụp, và cần **Rendering/Rasterisierung** mới hiển thị lên màn hình được.
- **SVG:**
  - Là XML dạng text: `<svg>`, `<rect x y width height fill>`, `<circle cx cy r>`, `<line x1 y1 x2 y2>`, `<polygon points="…">`, `<polyline>`, `<path d="M … Q … C …">`
  - Python: `svgwrite.Drawing("x.svg").add(dwg.circle(...))`, sau đó `.save()`
- ⭐ **Bézier bậc hai: B(t) = (1−t)²·P0 + 2(1−t)t·P1 + t²·P2**, với t ∈ [0, 1].
  - P0 và P2 là điểm đầu và cuối. **P1 là điểm điều khiển**, kéo cong đường nhưng đường cong **không đi qua** P1.
  - Bậc ba: `(1−t)³P0 + 3(1−t)²t·P1 + 3(1−t)t²·P2 + t³P3`.
- ⭐ **Transformationen (Matrizen):**
  - **Translation:** `p + (tx, ty)`
  - **Skalierung:** `S = [[sx, 0], [0, sy]]`
  - **Rotation:** `R = [[cos θ, −sin θ], [sin θ, cos θ]]`
  - Xoay hoặc phóng to **quanh tâm**: dời về gốc → biến đổi → dời lại về chỗ cũ
  - **Homogene Koordinaten** (ma trận 3×3) cho phép viết cả Translation dưới dạng nhân ma trận, nên **ghép được các phép biến đổi** (thứ tự quan trọng!)
- **Rendering / Rasterisierung:** chuyển vector thành pixel.

Unit 6: Interactive Graphics

- **Static vs. Interactive:**
  - Static: tính một lần, kết quả là một bức ảnh
  - Interactive: phản ứng theo input, **render real-time**, cập nhật liên tục
- ⭐ **Main Loop:**

```
Init → [ Input → Logic → Render ] lặp → Shutdown
```

Muốn 60 FPS thì mỗi vòng phải xong **dưới 16,6 ms**.

- ⭐ **Vấn đề framerate:** nếu di chuyển 2 cm mỗi frame thì 30 FPS đi 60 cm/s, 60 FPS đi 120 cm/s. Có hai cách giải quyết:
  - **Delta Time** (variable time step): `x += speed * delta`
  - **Fixed time step**: **tất định (deterministisch)**, quan trọng cho physics, networking, AI. Variable time step không tất định vì sai số làm tròn số thực.
- **Double Buffering:** vẽ vào **Back Buffer** rồi copy (blit/swap) sang **Primary Surface**. Nhờ đó không nhấp nháy.
- **Godot:**
  - Mã nguồn mở, khoảng 30 MB, không cần cài đặt.
  - **Node** có tên, thuộc tính, script, node con.
  - **Scene** là cây node có **đúng 1 root**, lưu thành `.tscn` (**packed scene**).
  - **Instancing:** dùng lại scene nhiều lần, có thể override thuộc tính cho từng instance.
  - **Main Scene** được đặt trong Project Settings.
- **GDScript:**
  - `extends Sprite2D` khai báo class cha. Không cần từ khóa `class`, mỗi file đã là một class.
  - `var` cho field, `func` cho hàm. Mỗi node có **đúng 1 script**.
  - **`_init()`** : chạy khi node được tạo
  - **`_ready()`** : chạy khi node đã vào scene tree
  - ⭐ **`_process(delta)`** : chạy **mỗi frame**, ví dụ `rotation += angular_speed * delta`
- **Signals:**
  - Node phát signal, node khác **connect** vào. Mỗi connection gồm signal nguồn và hàm đích.
  - Ví dụ: `timer.connect("timeout", self, "_on_Timer_timeout")` (cú pháp Godot 3).
  - Godot 4: `$Timer.timeout.connect(_on_timer_timeout)` .
- **Tìm node:**
  - `get_node("Timer")` hoặc `$Timer` : tìm một node
  - **Groups:** `add_to_group("enemies")` rồi `get_tree().get_nodes_in_group("enemies")` : tìm nhiều node
- ⭐ **Input:**
  - **Liên tục:** `Input.is_action_pressed("ui_left")` trong `_process` , dùng cho di chuyển
  - **Rời rạc:** `_unhandled_input(event) + event.is_action_pressed("jump")` , dùng cho nhấn một lần, click
  - `Input.is_action_just_pressed(...)` : true **đúng 1 frame**, dùng cho toggle

Unit 7: Animated Graphics

- **Persistence of Vision:** ảnh tĩnh chiếu nhanh liên tiếp tạo ảo giác chuyển động. **Frame** là từng ảnh, **FPS** là số frame mỗi giây.
- ⭐ **4 cách làm animation:**
  - **Frame-by-Frame:** vẽ từng frame (Sprite Sheet). Kiểm soát tối đa nhưng tốn công
  - **Keyframe:** máy tự nội suy (tween) phần giữa các keyframe
  - **Procedural:** dùng toán học (sin, noise, physics) theo thời gian thực
  - **State-based:** FSM với các state Idle/Run/Jump
- **Realtime vs. Offline:**
  - Realtime: game/UI, phải giữ FPS cao
  - Offline: phim, render trước, mỗi frame có thể mất hàng giờ
- **Sprite Sheet:**
  - Nhiều frame trong **một texture**, chọn frame bằng **UV-Koordinaten**
  - Giảm **Draw Calls** cho GPU
  - Godot: **AnimatedSprite2D** + **SpriteFrames**, gọi `play("walk")`
- **AnimationPlayer:**
  - Chứa các Animation, mỗi Animation có nhiều **Track**, mỗi Track điều khiển **một thuộc tính** (position, rotation, scale, modulate)
  - Có thể bật **loop**
  - Animation **RESET** lưu tư thế mặc định
- ⭐ **Tweening = in-betweening.**
  - **lerp:** $a + (b - a) \cdot t$ với $t \in [0, 1]$
  - **Easing:**
    - **Ease-In:** chậm rồi nhanh, t^2
    - **Ease-Out:** nhanh rồi chậm, $1 - (1 - t)^2$
    - **Smoothstep:** $3t^2 - 2t^3$
- ⭐ **Oscillation / Kreisbewegung:**
  - Con lắc: $x = \sin(time \cdot speed) \cdot amplitude$
  - Chuyển động tròn: $x = \cos(angle) \cdot r$, $y = \sin(angle) \cdot r$
- **FSM:**
  - **States:** từng animation
  - **Transitions:** đường chuyển giữa các state
  - **Triggers/Conditions:** ví dụ `velocity` , `is_on_floor()`
  - Godot: **AnimationTree** với root `AnimationNodeStateMachine` , gọi `["parameters/playback"].travel("Jump")`

⭐ TOP 15: NHẤT ĐỊNH PHẢI NHỚ

1. Dateigröße Audio = `Dauer × fs × Kanäle × Bit/8` . Raster = `B × H × Bit/8` .
2. Nyquist: `fs > 2·f_max` , nếu không sẽ bị Aliasing. **Samplerate quyết định dải tần, Bittiefe quyết định nhiễu/dải động.**
3. OpenCV: **BGR** (R = kênh 2), `shape = (H, W, C)` , `COLOR_BGR2GRAY` , `Dauer = frames / fps` .
4. `Y = 0.299R + 0.587G + 0.114B` . Green có trọng số cao nhất vì mắt nhạy nhất với xanh lá (và Bayer có 2G).
5. JPEG: **YCbCr → Subsampling → DCT → Quantisierung (lossy duy nhất) → RLE/Huffman**.
6. Container ≠ Codec. "WAV enthält PCM" là câu đúng.
7. FLAC = lossless, dùng cho archive. AAC hiệu quả hơn MP3. WAV dùng cho phân tích.
8. I/P/B-Frames. CRF 23 là mặc định. 4:2:0 giảm 50% dữ liệu.
9. Offset dùng `setpts` / `adelay` . Drift dùng `atempo` / `aresample` . `-c copy` khi cắt **không chính xác đến từng frame**.
10. J.248: **+45 ms** (Audio trước) / **−125 ms** (Audio sau). Thời lượng đúng không có nghĩa là đồng bộ đúng.
11. Spectral Centroid ≠ Pitch. Histogrammdifferenz phát hiện Shot, Frame-Differenz đo Bewegung. Score của YAMNet không phải xác suất.
12. `np.int32(arr + c).clip(0, 255).astype(np.uint8)` , vector hóa, không dùng vòng for.
13. Perlin: Grid + Gradient → Distanz → Dot → Fade `6t⁵ − 15t⁴ + 10t³` + lerp. Octave: Frequenz ×2, Amplitude ×½.
14. Mandelbrot: `z = z² + c` , `|z| > 2` thì thoát, `i` là số vòng lặp đến khi thoát, điểm trong tập tô đen.
15. Main Loop: Input → Logic → Render. `x += speed * delta` . `_process(delta)` chạy mỗi frame. Bézier: `(1−t)²P0 + 2(1−t)tP1 + t²P2` .